

Java Backend Architecture

**Interview Series**

# Chapter 14.1

## System Design Methodology

50+ Interview Q&A

Code Examples

Architecture Diagrams

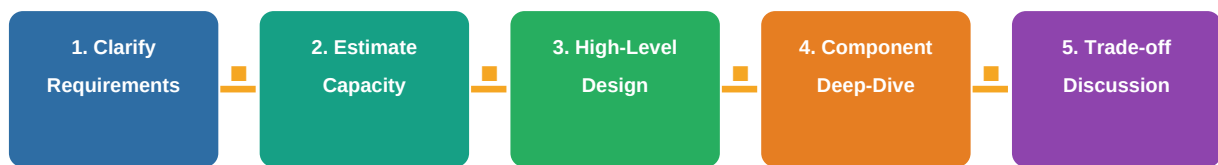
Advanced Topics

## Chapter 14.1 – System Design Methodology

### Overview

System design interviews test your ability to architect large-scale distributed systems. A structured 5-step framework helps you communicate clearly, cover all dimensions, and demonstrate senior-level thinking. This chapter covers the end-to-end methodology used by top engineers at FAANG and beyond.

### 5-Step Design Framework



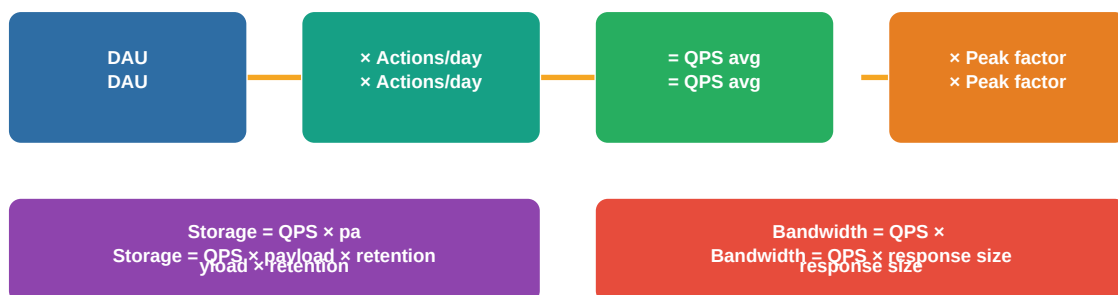
5-Step System Design Interview Framework

#### Step 1: Requirements Clarification

- Functional requirements: what the system does (APIs, features, user actions)
- Non-functional requirements: scalability, latency, availability, consistency, durability
- Constraints: budget, team size, technology stack, compliance (GDPR, HIPAA)
- Out of scope: clarify what you will NOT design to manage interview time
- Ask: read/write ratio, peak vs average load, geographic distribution

#### Step 2: Capacity Estimation

##### Capacity Estimation Formulas



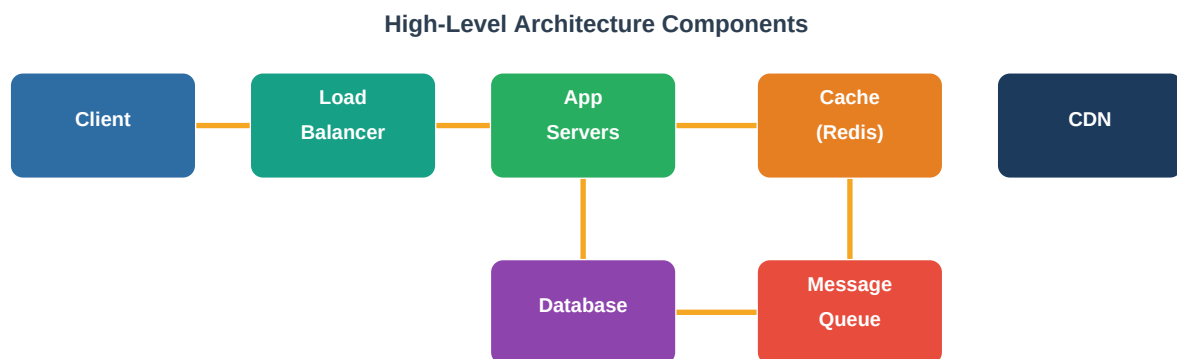
- DAU (Daily Active Users) × requests per user per day ÷ 86,400 = avg QPS
- Example: 1M DAU × 10 req/day = 10M req/day ÷ 86,400 ≈ 115 QPS average
- Peak = 3× average → 350 QPS; design for peak + 20% headroom
- Storage: QPS × avg payload size × retention period (seconds)
- Bandwidth: QPS × avg response size (read-heavy vs write-heavy matters)

- Cache: estimate hit ratio; size cache to absorb 80% of read traffic

### Back-of-Envelope Example — Social Feed

| Metric             | Calculation                    | Result              |
|--------------------|--------------------------------|---------------------|
| Daily Active Users | 1 million                      | 1,000,000           |
| Requests/user/day  | 10 actions                     | 10,000,000 req/day  |
| Avg QPS            | $10M \div 86,400$              | $\approx 115$ QPS   |
| Peak QPS (3×)      | $115 \times 3$                 | $\approx 350$ QPS   |
| Write ratio        | 20% of requests                | 70 writes/sec       |
| Avg post size      | 500 bytes                      | —                   |
| Storage/day        | $70 \times 500B \times 86,400$ | $\approx 3$ GB/day  |
| 1-year storage     | $3 \text{ GB} \times 365$      | $\approx 1$ TB/year |

### Step 3: High-Level Design



- Draw major components: clients, load balancer, app servers, DB, cache, CDN, queues
- Show data flow with arrows — request path from client to response
- Identify primary data stores and their responsibilities
- Group by domain: user service, post service, notification service

### Step 4: Component Deep-Dive

- Identify the bottleneck or most critical component — focus interviewer attention here
- Database schema: entity relationships, indexes, partitioning key choice
- API design: RESTful endpoints, request/response shapes, pagination
- Caching strategy: what to cache, TTL, invalidation, cache-aside vs write-through
- Async processing: which operations can be done out-of-band via queues

### Step 5: Trade-off Discussion

|                     |                                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------|
| <b>SQL vs NoSQL</b> | SQL: ACID, joins, schema. NoSQL: flexible schema, horizontal scale. Choose based on query patterns. |
|---------------------|-----------------------------------------------------------------------------------------------------|

|                                  |                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Strong vs Eventual</b>        | Strong: always reads latest write (higher latency). Eventual: higher availability, may read stale data.                  |
| <b>Monolith vs Microservices</b> | Monolith: simpler ops, single deployment. Microservices: independent scaling, polyglot, complex orchestration.           |
| <b>Sync vs Async</b>             | Sync: simpler error handling. Async (queues): decoupled, resilient, harder to debug ordering issues.                     |
| <b>Push vs Pull</b>              | Push (fan-out on write): fast reads, expensive writes for celebrities. Pull (fan-out on read): cheap writes, slow reads. |

## Non-Functional Requirements Checklist

| NFR           | Key Metric                 | Design Lever                             |
|---------------|----------------------------|------------------------------------------|
| Availability  | 99.9% = 8.7h downtime/yr   | Redundancy, health checks, failover      |
| Latency       | p99 < 100ms API            | Cache, CDN, async, DB indexing           |
| Scalability   | Handle 10× traffic spike   | Horizontal scale, auto-scaling, sharding |
| Consistency   | Read-your-writes guarantee | Sticky sessions, synchronous replication |
| Durability    | 11 nines (S3)              | Multi-AZ replication, WAL, backups       |
| Security      | Zero data breach           | Encryption at rest/transit, OAuth2, RBAC |
| Observability | MTTD < 5 min               | Metrics, logs, traces, alerting          |

## Interview Q&A; — System Design Methodology

### Q1. What are the 5 steps of a system design interview?

Requirements clarification (functional + non-functional), capacity estimation (DAU/QPS/storage), high-level design (components and data flow), component deep-dive (bottleneck focus), and trade-off discussion. Each step builds on the previous and signals structured senior-level thinking.

### Q2. What is the difference between functional and non-functional requirements?

Functional requirements define what the system does — user stories, API endpoints, features. Non-functional requirements define how well it performs — latency, availability, consistency, security, and scalability. Both must be clarified before designing.

### Q3. How do you estimate QPS from DAU?

$\text{QPS} = (\text{DAU} \times \text{requests\_per\_user\_per\_day}) \div 86,400 \text{ seconds}$ . For 1M DAU  $\times$  10 requests = 10M/day  $\div$  86,400  $\approx$  115 QPS average. Always multiply by 3 $\times$  for peak load to size infrastructure correctly.

### Q4. How do you estimate storage requirements?

$\text{Storage} = \text{write\_QPS} \times \text{avg\_payload\_bytes} \times \text{retention\_seconds}$ . For 70 writes/sec  $\times$  500 bytes  $\times$  86,400 sec/day  $\approx$  3 GB/day. Over a year: 3 GB  $\times$  365  $\approx$  1 TB. Add 20% overhead for indexes and metadata.

### Q5. Why is the bottleneck identification important in deep-dive?

Focusing on the bottleneck demonstrates senior judgment. If the bottleneck is the DB, dive into indexing, read replicas, and sharding. If it's the API layer, discuss rate limiting and caching. Spreading effort uniformly wastes precious interview time.

### Q6. What questions should you always ask the interviewer upfront?

How many users (DAU/MAU)? Read/write ratio? Consistency requirements? Available latency budget? Geographic distribution? Mobile vs web? Existing infrastructure constraints? These shape every subsequent design decision.

### Q7. How do you handle the read/write ratio in design?

Read-heavy systems (90:10 ratio) invest in caching (Redis, CDN), read replicas, and denormalized schemas. Write-heavy systems invest in buffered writes, message queues, async processing, and append-only data structures like Kafka logs.

### Q8. What is a back-of-envelope calculation?

A rapid order-of-magnitude estimate done without precise data, using known constants: 1 day = 86,400 sec, 1 MB =  $10^6$  bytes, 1 GB =  $10^9$  bytes. Goal is to identify whether you need 1 server or 100, not to get exact numbers. Accuracy within 2 $\times$  is acceptable.

### Q9. How do you choose between SQL and NoSQL?

SQL for: complex queries with joins, strict ACID transactions, well-defined schema, financial data. NoSQL for: flexible schema, massive scale, simple key-value or document access patterns, eventual consistency is acceptable (social feeds, product catalogs, session storage).

### Q10. What availability SLAs mean in practice?

99% = 87.6h downtime/year. 99.9% = 8.76h/year. 99.99% = 52.6 min/year. 99.999% = 5.26 min/year. Each additional nine requires significant investment in redundancy, failover, and operational maturity. Most B2C products target 99.9%-99.99%.

---

### Q11. How do you design for 10 $\times$ traffic spikes?

Stateless application tier with horizontal auto-scaling (CPU or request-based triggers). Pre-warm cache before events. Use CDN for static content (absorbs 70%+ of traffic). Async processing via queues smooths write bursts. Read replicas handle read spikes independently.

**Q12. What is the difference between latency and throughput?**

Latency is the time for a single request to complete (p50, p95, p99 percentiles). Throughput is the number of requests processed per second (QPS/TPS). They are related but different: high throughput systems can still have high latency under load. Design for both.

**Q13. How should you structure your high-level design diagram?**

Left-to-right flow: client → DNS/CDN → load balancer → app servers → data stores. Show read vs write paths separately if they differ. Add cache layer between app and DB. Show async path to message queue. Label components with technology choices (Nginx, Redis, PostgreSQL).

**Q14. What is capacity planning vs capacity estimation?**

Capacity estimation is done upfront in design: predicting resource needs from expected load. Capacity planning is ongoing operational work: monitoring actual utilization, provisioning ahead of growth, and adjusting based on real data. Interviews focus on estimation; real jobs require both.

**Q15. How do you identify the most critical component to deep-dive?**

Ask: where does failure hurt most? Where is data read/written most frequently? Where are performance bottlenecks likely? For URL shortener: the redirect DB. For Twitter: the feed fan-out. For Uber: the location tracking service. State your reasoning explicitly.

**Q16. What trade-offs should you always mention in system design?**

Consistency vs availability (CAP), latency vs durability, normalization vs denormalization, monolith vs microservices, synchronous vs asynchronous processing, cost vs performance, simplicity vs flexibility. Showing awareness of trade-offs signals architectural maturity.

**Q17. How do you handle the "design Twitter" type question?**

Clarify: DAU ~300M, 500M tweets/day. QPS: ~6K reads/sec, ~6K writes/sec. Focus on: tweet storage (NoSQL or sharded MySQL), fan-out on write vs read for timeline, Redis for hot timelines, Kafka for async fan-out. Deep-dive: fan-out service for celebrity accounts (hybrid approach).

**Q18. What is the 80/20 rule in caching?**

80% of traffic typically accesses 20% of data (Pareto principle). Cache that hot 20% to dramatically reduce database load. Size cache to hold the hot dataset. For a 1TB database with 20% hot data, a 200GB cache (e.g., Redis cluster) can serve 80% of reads from memory.

**Q19. How do you handle database schema design in an interview?**

Start with entities and relationships. Identify primary keys and foreign keys. Add indexes for query patterns (not all columns). Decide normalization level based on read/write ratio. For NoSQL: design by access patterns (single-table design in DynamoDB). Always explain your reasoning.

**Q20. What is the purpose of a load balancer in the design?**

Distributes traffic across multiple server instances to prevent any single server from becoming a bottleneck. Performs health checks to route away from failed instances. Enables zero-downtime deployments via rolling updates. Can terminate TLS, apply rate limiting, and route by path/header.

**Q21. How do you design an API for your system?**

Use RESTful conventions: POST /resources for create, GET /resources/{id} for read, PUT for full update, PATCH for partial update, DELETE for remove. Version APIs (v1/). Use pagination for list endpoints (cursor or offset). Return standard HTTP status codes. Document with OpenAPI/Swagger.

**Q22. What is the role of a CDN in system design?**

Content Delivery Network caches static assets (images, JS, CSS) at edge locations globally, reducing latency for end users and offloading origin server traffic. CloudFront, Cloudflare, Akamai are common. Can also cache dynamic API responses with short TTLs. Reduces bandwidth costs.

**Q23. How do you account for data growth in your design?**

Estimate storage at 1 year, 3 years, and 5 years. Plan data lifecycle: hot (SSD), warm (HDD), cold (S3 Glacier). Archive old data. Use sharding/partitioning strategy that allows horizontal growth without full re-architecture. Time-series data (logs, metrics) compresses 10:1.

**Q24. What is geo-distribution and when do you need it?**

Deploying infrastructure in multiple geographic regions to reduce latency for global users and provide disaster recovery. US East, EU West, Asia Pacific are common regions. Use Route 53 geolocation routing. Adds complexity: cross-region replication, data sovereignty compliance.

**Q25. How do you communicate trade-offs professionally in an interview?**

Use the "I chose X over Y because..." format. Acknowledge the downside: "The trade-off is that we sacrifice Z, which is acceptable because...". Offer alternatives: "If the requirements change to prioritize Z, I would switch to Y because...". This shows both knowledge and judgment.

**Q26. What is the difference between synchronous and asynchronous design?**

Synchronous: caller waits for response; simpler error handling, tight coupling. Asynchronous: caller publishes to queue and continues; decoupled, resilient to downstream failures, higher complexity. Use async for: notifications, emails, report generation, any operation where immediate response is not required.

**Q27. How do you estimate cache size?**

Cache size = hot\_data\_fraction × total\_dataset\_size × avg\_object\_size. For a product catalog with 1M products, 20% hot, avg 2KB/product: 200K × 2KB = 400MB — easily fits in Redis. Factor in key overhead (~50 bytes/key) and Redis overhead (~1.5×). Size Redis instances accordingly.

**Q28. What is horizontal partitioning (sharding)?**

Splitting rows of a table across multiple database servers based on a shard key (user\_id, tenant\_id, hash). Each shard holds a subset of rows. Enables linear scaling of write throughput and storage. Challenges: cross-shard queries, shard rebalancing, hot shards. Plan the shard key carefully — it's hard to change later.

**Q29. What are microservices and when should you suggest them?**

Independently deployable services each owning a bounded domain (user service, order service, payment service). Suggest when: system is large and complex, multiple teams work independently, different services need different scaling, polyglot persistence is valuable. Avoid for small systems — operational overhead exceeds benefits.

**Q30. How do you handle security requirements in a system design interview?**

Authentication: JWT or session tokens via OAuth2/OIDC. Authorization: RBAC or ABAC. Encryption: TLS 1.3 in transit, AES-256 at rest. Rate limiting: prevent DDoS and credential stuffing. Input validation: prevent SQL injection, XSS. Audit logs: track all sensitive operations. Mention GDPR compliance for EU user data.

**Q31. What metrics should you instrument on a new system?**

The RED method: Rate (requests/sec), Errors (error rate %), Duration (latency percentiles). The USE method for resources: Utilization, Saturation, Errors. Business metrics: active users, conversion rate, revenue. Infrastructure: CPU, memory, disk I/O, network. Use Prometheus + Grafana or Datadog for collection and visualization.

### Q32. How do you handle session management in a distributed system?

Avoid sticky sessions (server affinity) — breaks horizontal scaling. Instead: store sessions in Redis (shared cache) with TTL. Use JWT (stateless token) — no server-side state required but cannot invalidate before expiry. For sensitive apps (banking), server-side sessions in Redis with ability to invalidate are preferred.

### Q33. What is the difference between vertical and horizontal scaling?

Vertical scaling: add more CPU/RAM/disk to existing machine. Fast, simple, but has upper limit and single point of failure. Horizontal scaling: add more machine instances. Theoretically unlimited, provides redundancy, but requires stateless design and load balancing. Modern cloud architectures prefer horizontal scaling.

### Q34. How do you design for zero-downtime deployments?

Blue-green deployment: maintain two identical environments, switch DNS. Canary release: route 5% traffic to new version, monitor, then ramp up. Rolling update: replace instances one by one. Feature flags: deploy code but enable features gradually. Database migrations: backward-compatible schema changes (add before remove, never rename columns directly).

### Q35. What is the CAP theorem relevance in system design?

In a distributed system, you cannot simultaneously guarantee Consistency, Availability, and Partition tolerance. Since network partitions are unavoidable in distributed systems, you choose between CP (consistent during partition, may become unavailable) and AP (always available, may return stale data). This drives database and architecture choices.

### Q36. How do you handle large file uploads in your design?

Use pre-signed S3 URLs: client requests upload URL from server, uploads directly to S3 (bypasses your app servers), server receives S3 event notification. For chunked upload: split into 5-10MB chunks, upload in parallel, use S3 multipart upload API. Store metadata (filename, size, checksum, user\_id) in your DB, not the file itself.

### Q37. What is eventual consistency and when is it acceptable?

Eventual consistency means replicas will converge to the same value given no new updates. Acceptable for: social media likes/counts (approximate is fine), product recommendations, analytics dashboards, email delivery status. NOT acceptable for: bank balances, inventory counts, order processing, any financial transaction.

### Q38. How do you design rate limiting?

Token bucket algorithm: bucket holds N tokens, refills at R rate, each request consumes 1 token. Fixed window counter: count requests per minute per user. Sliding window: more accurate, prevents boundary bursts. Store counters in Redis with INCR + EXPIRE. Apply at load balancer (Nginx) or API gateway (Kong). Return 429 Too Many Requests with Retry-After header.

### Q39. What is the purpose of a message queue in system design?

Decouples producers from consumers: producer sends message and continues without waiting. Smooths traffic spikes: queue absorbs burst writes, consumers process at steady rate. Enables retry: failed messages can be requeued. Enables fan-out: one event triggers multiple consumers. Common use cases: async notifications, order processing, video transcoding.



**Q40. How do you present your system design solution clearly?**

Use a top-down approach: start with user interaction → front-end → API gateway → services → data stores. Draw as you speak. Narrate your decisions: "I'm choosing PostgreSQL here because...". Pause and check with interviewer: "Does this level of detail satisfy you, or should I go deeper?". Time management: spend 5 min on each of the 5 steps.

---

**Q41. What is the difference between a cache and a CDN?**

Cache (Redis/Memcached): in-memory key-value store inside your data center, speeds up DB queries, stores computed results, session data. CDN: geographically distributed edge network, caches static and cacheable dynamic content close to end users, reduces round-trip time globally. Both reduce latency but at different layers and for different content types.

**Q42. How do you design a notification system?**

Event source publishes to message queue (Kafka). Notification service consumes events, determines recipients and channel (push/email/SMS). Push: APNs for iOS, FCM for Android, WebSocket for web. Email: SendGrid/SES. SMS: Twilio. Store notification state (sent/read) in DB for inbox feature. Rate limit notifications per user to prevent spam.

**Q43. What is database denormalization and when do you use it?**

Deliberately introducing redundancy by storing pre-joined data to improve read performance. Example: storing username + avatar in both users table and posts table to avoid joins. Use when: read/write ratio is high (10:1+), join is expensive, strict ACID not required. Trade-off: write complexity increases (must update all copies), storage increases.

**Q44. How do you handle the "design a URL shortener" question systematically?**

Requirements: shorten URLs, redirect, analytics. Scale: 100M URLs/day. QPS: 1157 writes/sec, 11,570 reads/sec (10:1 read ratio). Key: base62 encode 7 chars =  $62^7 = 3.5T$  unique URLs. DB: NoSQL (key: short\_code, value: long\_url). Cache: Redis for hot URLs. Redirect: 301 (cached by browser) vs 302 (analytics-friendly). CDN: cache redirects globally.

**Q45. What is a service mesh and when do you mention it?**

Infrastructure layer for service-to-service communication in microservices. Handles: mutual TLS, service discovery, load balancing, circuit breaking, observability (distributed tracing). Istio (Envoy sidecar) and Linkerd are common. Mention when: large microservices architecture, East-West traffic security is a requirement, observability across services is needed.

**Q46. How do you handle distributed transactions?**

Two-phase commit (2PC): coordinator asks all participants to prepare, then commits. Simple but blocking. Saga pattern: sequence of local transactions with compensating transactions on failure. Choreography-based saga uses events. Orchestration-based uses a saga orchestrator. For most systems, sagas with eventual consistency are preferred over 2PC for scalability.

**Q47. What is the idempotency pattern and why is it important?**

An idempotent operation produces the same result regardless of how many times it is called. Critical for: payment processing (prevent double charges), order creation (prevent duplicate orders), API retries. Implement with: idempotency key in request header, store key + result in DB, check before processing. Return cached result if key already exists.

**Q48. How do you design for fault tolerance?**

Redundancy: multiple instances of each component. Health checks + auto-restart. Circuit breaker: stop calling failing services (Resilience4j). Retry with exponential backoff + jitter. Bulkhead: isolate failures (separate thread pools per dependency). Timeout: don't wait indefinitely. Graceful degradation: serve partial results when dependencies fail.

**Q49. What is observability and how do you design for it?**

Observability: ability to understand system state from its outputs. Three pillars: Metrics (Prometheus/Datadog for numerical time-series), Logs (structured JSON logs via ELK stack or Splunk), Traces (distributed request tracing with Jaeger/Zipkin/AWS X-Ray). Correlation IDs propagated through all service calls enable end-to-end trace reconstruction.

**Q50. How do you handle pagination in API design?**

Offset pagination: `?page=2&size=20` — simple, allows jumping to any page, but slow for large offsets (DB must scan all preceding rows). Cursor pagination: `?cursor=eyJ...` — efficient for large datasets, only forward navigation, stable across inserts. Keyset pagination: `?after_id=12345` — uses indexed column, very efficient. Choose cursor/keyset for large data.

**Q51. What is the strangler fig pattern?**

Incrementally migrate a monolith to microservices by "strangling" the monolith over time. New features built as microservices, legacy functionality gradually extracted. API gateway or facade routes traffic to old monolith or new services. Zero big-bang rewrite risk. Named after the strangler fig tree that gradually envelops and replaces its host tree.